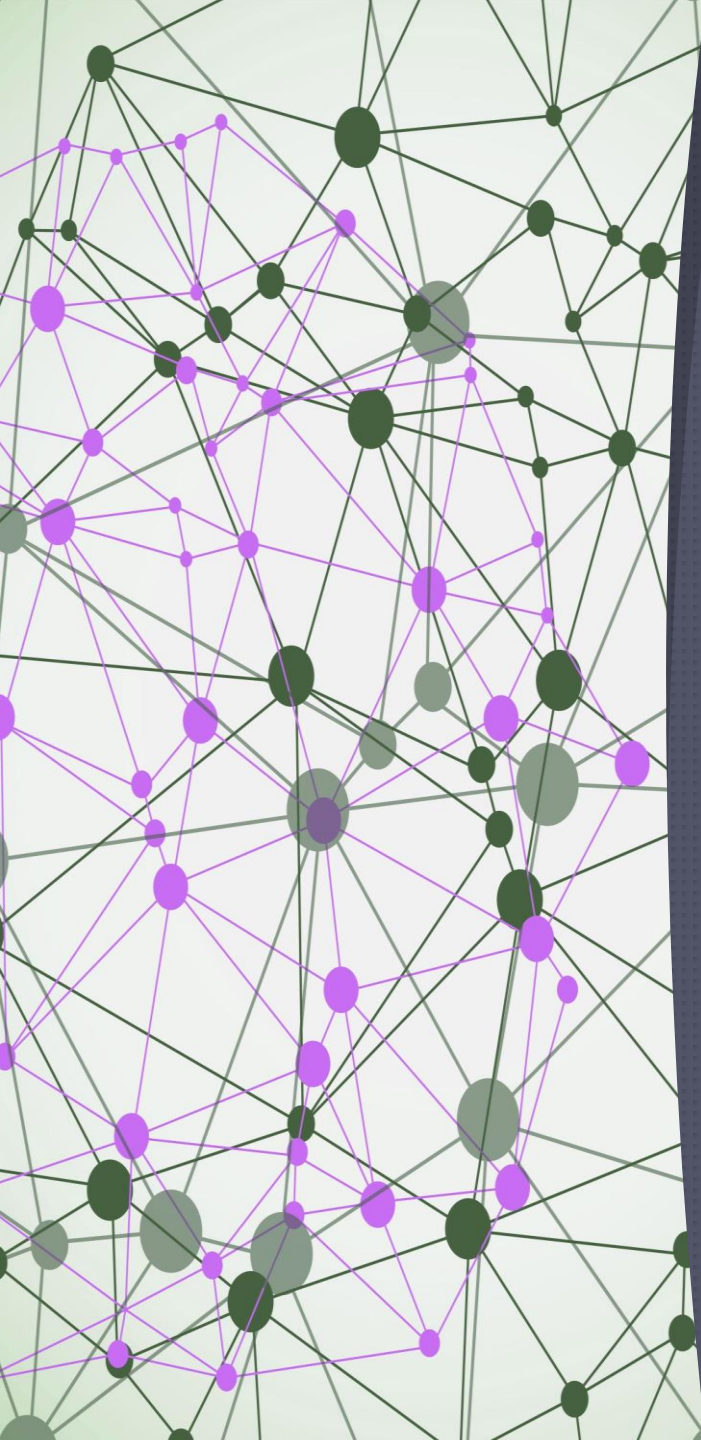




JavaScript

...Suite

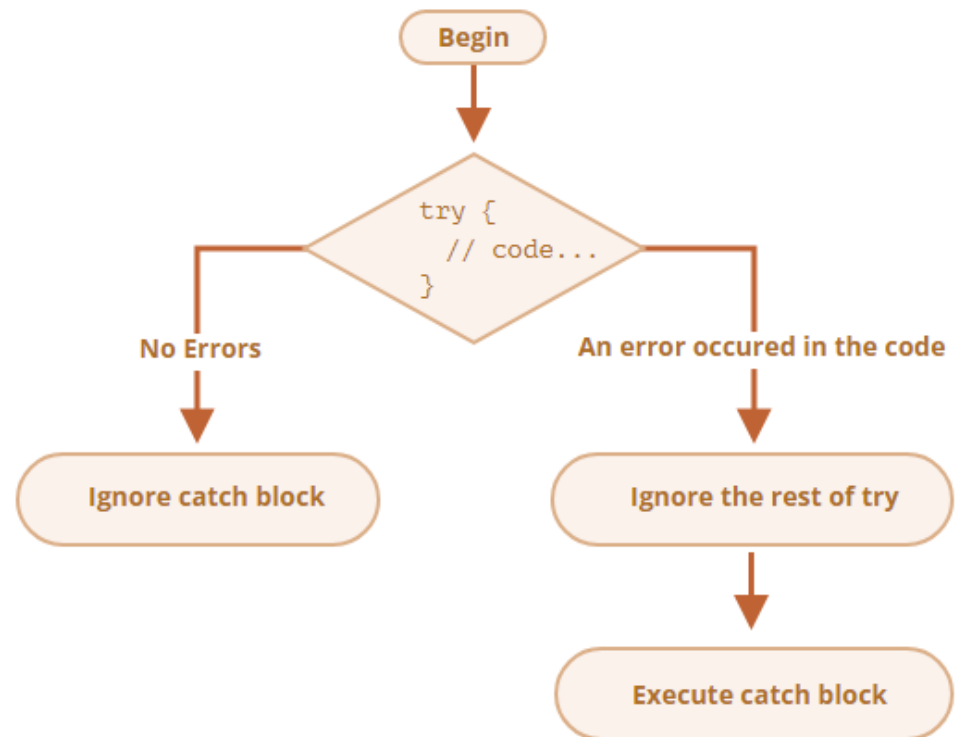
Gestion des erreurs

- Les erreurs d'un script peuvent être dues :
 - à nos erreurs
 - à une entrée utilisateur imprévue
 - à une réponse erronée du serveur
 - à d'autres raisons.
- Généralement, un script s'arrête immédiatement en cas d'erreur en l'affichant dans la console.
- La structure **try...catch** empêche l'arrêt du script en exécutant d'autres instructions en cas d'erreur.

Gestion des erreurs

- La structure **try...catch** fonctionne comme-ci :

```
try {  
    // code...  
} catch (err) {  
    // Gestion des erreurs  
}
```



Gestion des erreurs

- Exemple :

L'objet **erreur** a trois propriétés principales :

- **Name** : Nom de l'erreur.
- **Message** : Message textuel sur les détails de l'erreur.
- **Stack** : Chaîne contenant des informations supplémentaires sur l'erreur.

```
try{  
    alert('Bonjour : ' + prenom);  
}catch(err){  
    alert('Erreur rencontrée. ' +  
        '\nNom de l\'erreur : ' + err.name +  
        '\nMessage d\'erreur : ' + err.message +  
        '\nEmplacement de l\'erreur : ' + err.stack);  
}
```

Erreur rencontrée.
Nom de l'erreur : ReferenceError
Message d'erreur : prenom is not defined
Emplacement de l'erreur : ReferenceError: prenom is not defined
at http://127.0.0.1:5500/test4.html:191:25

Gestion des erreurs

- Remarques:
 - try...catch **ne fonctionne que pour les erreurs d'exécution** (syntaxe correcte)
 - try...catch **fonctionne de manière synchrone**

Gestion des exceptions

- Parfois, on sait à l'avance que notre code va provoquer une erreur dans certains cas d'utilisation. (par exemple division par zéro)
- Dans ce cas, l'instruction **throw** permet de générer un message d'erreur personnalisé au lieu de laisser JavaScript génère son propre message.

Gestion des exceptions

- Exemple :

```
try{
  let x = prompt('Entrez un premier nombre (numérateur)');
  let y = prompt('Entrez un deuxième nombre (dénominateur)');

  if(y == 0){
    throw new Error('Division par 0 impossible')
  }else {
    alert(x / y);
  }
}catch(err){
  alert(err.message);
}
```

Promesse

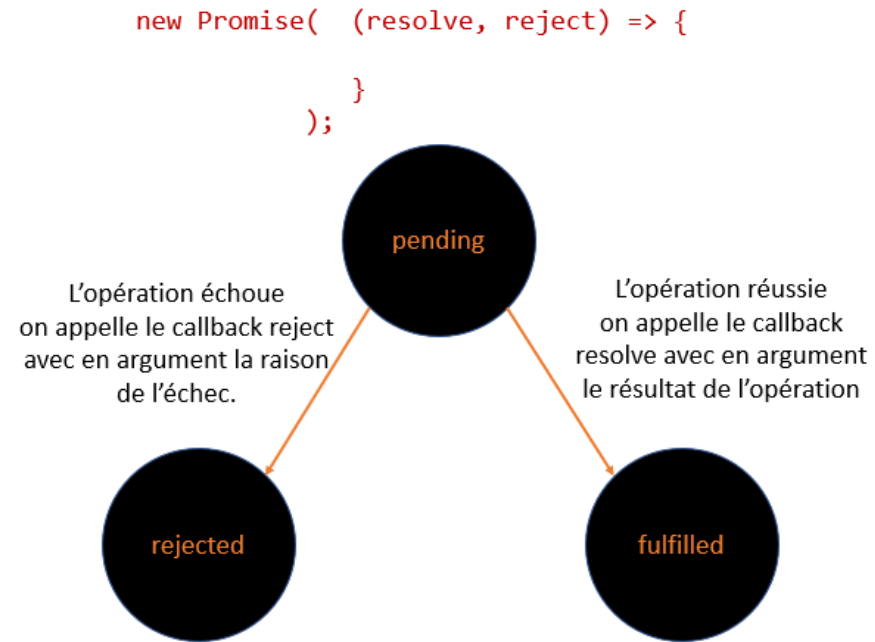
Les promesses (Promise)

- Les promesses sont des objets qui retournent une valeur future d'une opération asynchrone. Elles disposent de méthodes permettant de traiter le résultat une fois l'opération accomplie (`then()` et `catch()`).

Promesse

Il existe 3 états pour une promesse :

- En cours (pending) : la valeur contenue dans la promesse n'est pas encore arrivée.
- Résolue (fulfilled) : la valeur de la promesse est arrivée et est utilisable. La fonction `resolve()` est appelée.
- Rejetée (rejected) : une erreur est survenue et il est possible d'effectuer une action en réaction à cette erreur. La fonction `reject()` est appelée.



Promesse

- Comment fonctionne une Promesse ?

```
let maPromesse = new Promise((resolve, reject) => {  
  let toutVaBien = true;  
  
  if(toutVaBien) {  
    resolve('La promesse est résolue !');  
  } else {  
    reject('La promesse est rejetée.');  }  
});
```

Promesse

- Comment utiliser une Promesse ?

```
maPromesse
    .then(message => {
        console.log(message);
    })
    .catch(erreur => {
        console.log(erreur);
    });
```

Promesse

- Exemple :

```
let promis = new Promise ((resolve, reject) => {  
  var xhr = new XMLHttpRequest();  
  xhr.open('GET', 'file1.txt');  
  xhr.send();  
  xhr.addEventListener('readystatechange', function() {  
    // On gère ici une requête asynchrone  
    if (xhr.readyState == XMLHttpRequest.DONE )  
      if (xhr.status == 200)  
        resolve('réussie '+xhr.responseText)  
      else  
        reject('échouée '+xhr.responseText)  
  });  
});  
promis.then(ok => { alert(ok) })  
promis.catch(err => { alert(err) })
```

Exercices

Exercice 1 :

Demandez à l'utilisateur de saisir un nombre. Essayez de convertir cette saisie en entier. Si l'utilisateur entre une valeur non numérique attrapez l'exception et affichez un message personnalisé d'erreur. Gérez cette erreur dans un bloc try/catch.

Exercices

Exercice 2 :

1. Créez une fonction `checkAge` qui prend un âge en paramètre et retourne une Promesse :

- Si l'âge est supérieur ou égal à 18, la Promesse doit se résoudre avec le message "Accès autorisé".
- Sinon, la Promesse doit être rejetée avec le message "Accès refusé".

2. Testez votre fonction en appelant `checkAge(20)` et en gérant le résultat :

- Utilisez `.then()` pour afficher la réponse en cas de succès ("Accès autorisé").
- Utilisez `.catch()` pour afficher l'erreur en cas de rejet ("Accès refusé" si l'âge est inférieur à 18).